

HTTP/2 Protocol

Introduction to HTTP/2

- **Standardization:** HTTP/2 [RFC 7540] was standardized in 2015. HTTP/1.1 was standardized in 1997.
- **Adoption:** By 2020, over 40% of the top 10 million websites support HTTP/2 [W3Techs]. Major browsers such as Chrome, Safari, Firefox, Opera, and Internet Explorer support HTTP/2.
- **Primary Goals:**
 - Reduce perceived latency using request/response multiplexing over a single TCP connection.
 - Enable request prioritization and server push.
 - Efficient compression of HTTP header fields.
- **Note:** HTTP methods, status codes, URLs, and headers remain unchanged. Only the data transport and formatting are modified.

Motivation for HTTP/2

Head-of-Line (HOL) Blocking in HTTP/1.1

- HTTP/1.1 uses persistent TCP connections to send a web page over a single TCP connection.
- **HOL Blocking Problem:** A large object (e.g., video) at the head of the connection can block smaller objects behind it.
- Browsers work around HOL by opening multiple parallel TCP connections (up to 6), both to reduce HOL effects and to gain more bandwidth due to TCP congestion control.



Figure 1: HOL Blocking in HTTP/1.1 – large object blocks smaller objects.

TCP Congestion Control Note: Each TCP connection gets roughly an equal share of a bottleneck link. Multiple connections allow browsers to "grab" more bandwidth for a single page.

HTTP/2 Goal

- Reduce or eliminate the need for multiple parallel TCP connections for a single web page.
- Allows TCP congestion control to function as intended.
- Requires mechanisms to avoid HOL blocking even with one persistent connection.

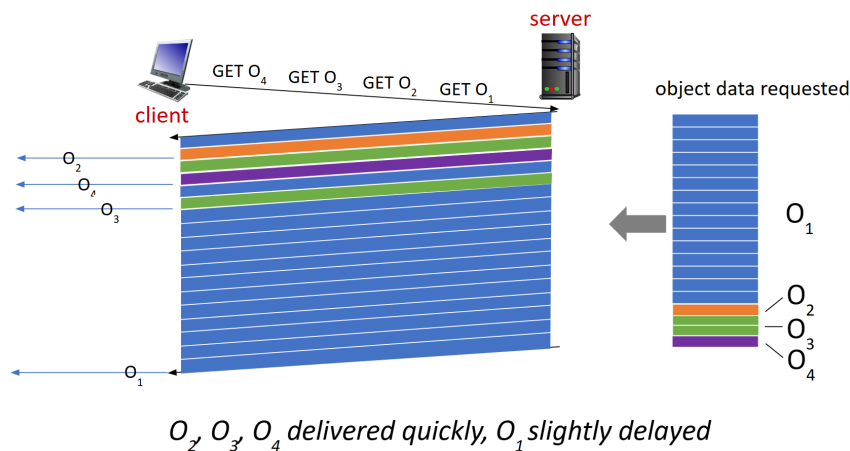


Figure 2: HTTP/2 Frame Interleaving – small objects can be delivered without waiting for large objects.

HTTP/2 Framing Mechanism

- **Solution for HOL Blocking:** Break each HTTP message into small frames and interleave them on the same TCP connection.
- Example: One large video (1000 frames) + 8 smaller objects (2 frames each):
 - Interleaving sends 1 video frame, then 1 frame of each small object, repeating until complete.
 - Without interleaving, small objects would be delayed until all 1016 frames of the video + objects are sent.
- **Frame Advantages:** Reduces user-perceived delay and enables efficient multiplexing.

Framing Sub-Layer

- Splits HTTP messages into frames: headers into one frame, body into multiple frames.

- Interleaves frames of different responses before sending over a single TCP connection.
- Frames are binary encoded: easier parsing, slightly smaller frames, less error-prone.
- At the client side, frames are reassembled into original HTTP messages.

Response Prioritization and Server Push

Message Prioritization

- Clients can assign a weight (1–256) to each request to indicate priority.
- Messages can have dependencies; a message can specify which other message it depends on.
- Server schedules frame transmission according to priority and dependencies.

Server Push

- Server can proactively send additional objects related to a client request.
- Example: HTML base page indicates objects required to render the page.
- Eliminates latency of waiting for client requests for each object.

Summary

- HTTP/2 reduces latency via multiplexing and frame interleaving.
- Eliminates or reduces multiple parallel TCP connections per page.
- Frames are binary encoded for efficiency and error reduction.
- Prioritization allows fine-grained control over response delivery.
- Server push reduces additional request latency.